

ER model a návrh databáze

Co je ER model

- **ER model** = Entity-Relationship model
- česky: entitně-relační model
- používá se při návrhu databáze
- pomáhá popsat:
 - jaká data chceme ukládat
 - jaké objekty v systému existují
 - jaké mají vlastnosti
 - jak spolu souvisí
- výsledkem bývá **ER diagram**

ER model se používá hlavně v počáteční fázi návrhu databáze.

ER diagram

- **ER diagram** = grafické znázornění ER modelu
- ukazuje entity, atributy a vztahy mezi entitami
- pomáhá pochopit strukturu databáze ještě před vytvořením tabulek
- obvykle se kreslí pomocí:
 - obdélníků
 - čar
 - popisků
 - značek kardinality

Zjednodušený příklad:

Student — patří do — Třída

Význam:

- existuje entita Student
- existuje entita Třída
- mezi nimi je vztah

Úrovně návrhu databáze

1. Konceptuální model

- řeší, **co** chceme ukládat

- neřeší technické detaily
- nezajímá nás konkrétní databázový systém
- popisuje objekty reálného světa a jejich vztahy
- typicky se kreslí jako ER diagram

Příklad:

- evidujeme studenty
- studenti patří do tříd
- učitelé učí předměty

V této fázi řešíme hlavně:

- entity
- atributy
- vztahy

2. Logický model

- převádí konceptuální návrh do podoby tabulek
- řeší už databázovou strukturu
- entity se mění na tabulky
- atributy se mění na sloupce
- vztahy se převádí pomocí klíčů

V této fázi řešíme:

- tabulky
- sloupce
- primární klíče
- cizí klíče
- vazby 1:1, 1:N, M:N
- spojovací tabulky

Příklad:

```
Student(student_id, jmeno, prijmeni, trida_id)
Trida(trida_id, nazev)
```

3. Fyzický model

- konkrétní implementace pro vybraný databázový systém
- řeší technické detaily
- závisí na konkrétní databázi

Příklady systémů:

- MySQL
- PostgreSQL
- Microsoft SQL Server
- Oracle Database
- SQLite

Ve fyzickém modelu řešíme:

- datové typy
- indexy
- omezení
- názvy tabulek a sloupců
- výkon
- fyzické uložení dat

Příklad:

```
CREATE TABLE Student (  
    student_id INT PRIMARY KEY,  
    jmeno VARCHAR(50) NOT NULL,  
    prijmeni VARCHAR(50) NOT NULL,  
    trida_id INT  
);
```

Změna názvosloví při návrhu

Při přechodu z ER modelu do relační databáze se mění názvy pojmů.

ER model	Relační databáze
entita	tabulka
atribut	sloupec
instance entity	řádek / záznam
relace / vztah	vazba přes klíče
identifikátor	primární klíč

Příklad:

- entita `Student` se změní na tabulku `Student`
- atribut `Jméno` se změní na sloupec `jmeno`
- konkrétní student Jan Novák se změní na jeden řádek v tabulce

Entita

Co je entita

- **entita** = typ objektu z reálného světa, který chceme evidovat
- obvykle odpovídá podstatnému jménu
- v ER diagramu se často kreslí jako obdélník
- entita není konkrétní záznam, ale obecný typ

Příklady entit:

- Student
- Třída
- Učitel
- Kniha
- Autor
- Objednávka
- Produkt
- Zaměstnanec

Entita vs. instance entity

Entita

- obecný typ objektu

Příklad:

Student

Instance entity

- konkrétní výskyt entity

Příklad:

Jan Novák
Eva Svobodová
Petr Dvořák

V databázi:

- entita `Student` bude tabulka

- instance Jan Novák bude jeden řádek

Atribut

Co je atribut

- **atribut** = vlastnost entity
- popisuje, jaké informace o entitě ukládáme
- v databázi se z atributu stane sloupec

Příklad entity Student :

- student_id
- jmeno
- prijmeni
- datum_narozeni
- email
- trida_id

Příklad entity Kniha :

- kniha_id
- nazev
- isbn
- rok_vydani

Typy atributů

Jednoduchý atribut

- nedělí se dál na menší části

Příklad:

- věk
- cena
- počet kusů

Složený atribut

- dá se rozdělit na menší části

Příklad:

Jméno = křestní jméno + příjmení
Adresa = ulice + město + PSČ

V relační databázi se často rozděluje do více sloupců:

```
first_name, last_name  
ulice, mesto, psc
```

Vícehodnotový atribut

- jedna entita může mít více hodnot stejného atributu

Příklad:

- člověk má více telefonních čísel
- student má více e-mailů

V relační databázi se většinou řeší samostatnou tabulkou.

Příklad:

```
Student(student_id, jmeno)  
StudentTelefon(student_id, telefon)
```

Relace / vztah

Co je relace

- **relace** v ER modelu znamená vztah mezi entitami
- říká, jak spolu entity souvisí
- v diagramu se kreslí jako čára mezi entitami
- v databázi se realizuje pomocí klíčů

Příklady vztahů:

- student patří do třídy
- autor napsal knihu
- zákazník vytvořil objednávku
- zaměstnanec má nadřízeného
- učitel učí předmět

Pozor na názvosloví:

- v ER modelu se slovem relace často myslí vztah mezi entitami
- v relačním modelu může relace znamenat tabulku

Klíče

Proč používáme klíče

- slouží k jednoznačné identifikaci záznamů
- propojují tabulky mezi sebou
- pomáhají zajistit integritu dat

Kandidátní klíč

- **kandidátní klíč** = atribut nebo kombinace atributů, která jednoznačně identifikuje řádek
- z kandidátních klíčů se vybírá primární klíč
- kandidátní klíč musí být jednoznačný
- měl by být minimální

Příklady kandidátních klíčů:

- student_id
 - e-mail
 - rodné číslo
 - ISBN knihy
 - číslo občanského průkazu
-

Přirozený klíč

- **přirozený klíč** = klíč, který vychází z reálných dat
- existuje i mimo databázi

Příklady:

- rodné číslo
- e-mail
- ISBN
- číslo občanského průkazu

Nevýhody:

- může se změnit
 - může být citlivý údaj
 - nemusí být vždy dostupný
 - někdy nemusí být opravdu unikátní
 - může být dlouhý nebo nepraktický
-

Umělý klíč

- **umělý klíč** = klíč vytvořený jen pro potřeby databáze
- nemá význam mimo databázi
- často se používá jako primární klíč

Příklady:

- `id`
- číselná sekvence
- GUID / UUID

Příklad:

```
Student(id, jmeno, prijmeni, email)
```

Výhoda:

- jednoduchý
- stabilní
- neobsahuje citlivé údaje
- dobře se používá ve vazbách

Primární klíč

- **primární klíč** = vybraný kandidátní klíč
- označuje se **PK**
- jednoznačně identifikuje řádek v tabulce
- nesmí být `NULL`
- nesmí se opakovat
- často se používá umělý číselný klíč

Příklad:

```
Student(student_id, jmeno, prijmeni)
```

- `student_id` je primární klíč

Cizí klíč

- **cizí klíč** = atribut, který odkazuje na primární klíč jiné tabulky
- označuje se **FK**
- fyzicky realizuje vztah mezi tabulkami
- zajišťuje referenční integritu

Příklad:


```
Trida(trida_id, nazev)
Student(student_id, jmeno, trida_id)
```

- `Student.trida_id` je cizí klíč
- odkazuje na `Trida.trida_id`

Význam:

- student může patřit jen do existující třídy

Kardinalita vztahu

Co je kardinalita

- **kardinalita** určuje, kolik záznamů jedné entity může souviset s kolika záznamy druhé entity
- popisuje početnost vztahu mezi entitami

Základní typy:

- 1:1
- 1:N
- M:N

Někdy se používá zápis:

- 1
- 0..1
- 0..N
- 1..N

Vztah 1:1

Co je vztah 1:1

- jednomu záznamu v první tabulce odpovídá nejvýše jeden záznam v druhé tabulce
- zároveň jednomu záznamu ve druhé tabulce odpovídá nejvýše jeden záznam v první tabulce
- používá se méně často

Příklad:

- člověk má jeden občanský průkaz
- občanský průkaz patří jednomu člověku

Realizace vztahu 1:1

Možnost 1:

- dát cizí klíč do jedné z tabulek
- tento cizí klíč by měl být zároveň `UNIQUE`

Příklad:

```
Clovek(clovek_id, jmeno)
Obcanka(obcanka_id, cislo, clovek_id UNIQUE)
```

Možnost 2:

- pokud mají entity vždy existovat spolu, mohou se sloučit do jedné tabulky

Příklad:

```
Clovek(clovek_id, jmeno, cislo_obcanky)
```

Vztah 1:N

Co je vztah 1:N

- jednomu záznamu v první tabulce odpovídá více záznamů v druhé tabulce
- jeden záznam ve druhé tabulce patří právě jednomu záznamu v první tabulce
- nejběžnější typ vazby

Příklad:

- jedna třída má více studentů
- jeden student chodí do jedné třídy

Realizace vztahu 1:N

- cizí klíč se dává na stranu `N`
- tedy do tabulky, kde je více záznamů

Příklad:

```
Trida(trida_id, nazev)
Student(student_id, jmeno, trida_id)
```

Význam:

- `Student.trida_id` odkazuje na `Trida.trida_id`
- jedna třída může mít mnoho studentů
- student má jednu třídu

Vztah M:N

Co je vztah M:N

- jednomu záznamu v první tabulce odpovídá více záznamů ve druhé tabulce
- zároveň jednomu záznamu ve druhé tabulce odpovídá více záznamů v první tabulce

Příklad:

- autor může napsat více knih
- kniha může mít více autorů

Další příklady:

- student navštěvuje více předmětů
- předmět má více studentů
- objednávka obsahuje více produktů
- produkt může být ve více objednávkách

Realizace vztahu M:N

- relační databáze neumí M:N přímo uložit
- vytvoří se třetí tabulka
- říká se jí:
 - spojovací tabulka
 - vazební tabulka
 - asociační tabulka
- obsahuje cizí klíče na obě původní tabulky
- M:N se rozpadne na dvě vazby 1:N

Příklad:

```
Autor(autor_id, jmeno)
Kniha(kniha_id, nazev)
AutorKniha(autor_id, kniha_id)
```

Význam:

- `AutorKniha.autor_id` odkazuje na `Autor.autor_id`
- `AutorKniha.kniha_id` odkazuje na `Kniha.kniha_id`

Spojovací tabulka s vlastním atributem

- Spojovací tabulka může obsahovat i další informace o vztahu.

Příklad:

```
Objednavka(objednavka_id, datum)
Produkt(produkt_id, nazev)
ObjednavkaProdukt(objednavka_id, produkt_id, mnozstvi,
cena_v_dobe_objednavky)
```

Význam:

- `mnozstvi` patří ke vztahu objednávky a produktu
- `cena_v_dobe_objednavky` je důležitá, protože cena produktu se může později změnit

Speciální vztahy

Rekurzivní vztah

- entita má vztah sama k sobě
- tabulka obsahuje cizí klíč, který odkazuje do stejné tabulky

Příklad:

- zaměstnanec má nadřízeného
- nadřízený je také zaměstnanec

Realizace:

```
Zamestnanec(zamestnanec_id, jmeno, nadrizeny_id)
```

Význam:

- `nadrizeny_id` odkazuje na `Zamestnanec.zamestnanec_id`
- jeden zaměstnanec může mít jednoho nadřízeného
- jeden nadřízený může mít více podřízených

ISA vztah

- vyjadřuje generalizaci a specializaci
- podobá se dědičnosti v objektovém programování
- znamená „je typem“

Příklad:

```
Student ISA Clovek
Ucitel ISA Clovek
```

Význam:

- student je člověk

- učitel je člověk

Realizace ISA vztahu

1. Jedna společná tabulka

```
Clovek(clovek_id, jmeno, prijmeni, typ, trida, kabinet)
```

Výhoda:

- jednoduché dotazy

Nevýhoda:

- hodně NULL hodnot
- tabulka může být nepřehledná

2. Tabulka pro předka a tabulky pro potomky

```
Clovek(clovek_id, jmeno, prijmeni)
Student(clovek_id, trida)
Ucitel(clovek_id, kabinet)
```

Výhoda:

- čistší návrh
- méně prázdných sloupců
- dobře odpovídá dědičnosti

Nevýhoda:

- častěji je potřeba používat JOIN

Postup návrhu databáze podle zadání

1. Přechíst zadání

- najít podstatná jména
- často odpovídají entitám

Příklad zadání:

Škola eviduje studenty, třídy, učitele a předměty.

Možné entity:

- Student
- Třída
- Učitel
- Předmět

2. Najít atributy

- ke každé entitě určit vlastnosti
- neukládat všechno zbytečně
- ukládat jen to, co systém potřebuje

Příklad:

```
Student(jmeno, prijmeni, datum_narozeni, email)
Trida(nazev, rocnik)
Ucitel(jmeno, prijmeni, kabinet)
Predmet(nazev)
```

3. Najít klíče

- každé tabulce dát primární klíč
- často umělý klíč `id`

Příklad:

```
Student(student_id, jmeno, prijmeni, datum_narozeni, email)
Trida(trida_id, nazev, rocnik)
Ucitel(ucitel_id, jmeno, prijmeni, kabinet)
Predmet(predmet_id, nazev)
```

4. Najít vztahy

Ptáme se:

- Patří student do třídy?
- Učí učitel předmět?
- Může mít předmět více učitelů?
- Může student chodit na více předmětů?

5. Určit kardinalitu

Příklad:

- Třída 1:N Student
- Učitel M:N Předmět

- Student M:N Predmet

6. Převést vztahy do tabulek

1:N

- cizí klíč jde na stranu N

```
Student(..., trida_id)
```

M:N

- vytvoří se spojovací tabulka

```
UcitelPredmet(ucitel_id, predmet_id)  
StudentPredmet(student_id, predmet_id)
```

1:1

- cizí klíč s UNIQUE
- nebo sloučení tabulek

7. Zkontrolovat návrh

Kontrolujeme:

- má každá tabulka primární klíč?
- jsou vztahy vyřešené cizími klíči?
- nejsou někde seznamy v jednom sloupci?
- není někde zbytečná duplicita?
- je M:N vyřešená spojovací tabulkou?
- dávají vazby smysl podle zadání?

Praktický příklad návrhu

Zadání

Škola eviduje studenty, třídy a předměty.

- Každý student patří právě do jedné třídy.
- Jedna třída má více studentů.
- Student může navštěvovat více předmětů.
- Jeden předmět může navštěvovat více studentů.

Entity

- Student
- Třída
- Předmět

Vztahy

- Třída 1:N Student
- Student M:N Předmět

Logický návrh tabulek

```
Trida(trida_id, nazev)
Student(student_id, jmeno, prijmeni, trida_id)
Predmet(predmet_id, nazev)
StudentPredmet(student_id, predmet_id)
```

Vysvětlení:

- `Student.trida_id` řeší vazbu 1:N mezi třídou a studentem
- `StudentPredmet` řeší vazbu M:N mezi studentem a předmětem

Praktický příklad s rekurzí, 1:1 a M:N

Zadání

Firma eviduje zaměstnance, jejich občanské průkazy, projekty a nadřízené.

- Každý zaměstnanec může mít jednoho nadřízeného.
- Nadřízený je také zaměstnanec.
- Každý zaměstnanec má jeden občanský průkaz.
- Jeden občanský průkaz patří jednomu zaměstnanci.
- Zaměstnanec může pracovat na více projektech.
- Na jednom projektu může pracovat více zaměstnanců.

Entity

- Zaměstnanec
- Občanský průkaz
- Projekt

Vztahy

- Zaměstnanec 1:N Zaměstnanec
 - rekurzivní vztah
 - nadřízený a podřízený

- Zaměstnanec 1:1 Občanský průkaz
- Zaměstnanec M:N Projekt

Logický návrh tabulek

```
Zamestnanec(zamestnanec_id, jmeno, prijmeni, nadrizeny_id)
ObcanskyPrukaz(obcanka_id, cislo, zamestnanec_id UNIQUE)
Projekt(projekt_id, nazev)
ZamestnanecProjekt(zamestnanec_id, projekt_id)
```

Vysvětlení:

- `Zamestnanec.nadrizeny_id` odkazuje zpět na `Zamestnanec.zamestnanec_id`
- `ObcanskyPrukaz.zamestnanec_id` `UNIQUE` řeší vztah 1:1
- `ZamestnanecProjekt` řeší vztah M:N